Robot Dynamics for Embodied AI: Theory and Simulation in Practice

Concise revised edition | From policy actions to contact-rich manipulation, simulation, and Sim2Real | No code

2026-10-30

Contents

1. Understanding Dynamics Through a Single Grasp . . . 2
2. Actuators: Why a Position Command Does Not Take Effect Instantly . . . 2
3. Mass, Inertia, and Gravity: Start With One Joint . . . 3
4. Multi-Joint Robots: One Equation Explains the Coupling . . . 4
5. The Jacobian: Connecting Fingertip Motion and Joint Forces . . . 4
6. Contact and Friction: Why Objects Stay Grasped or Slip . . . 5
7. From Dynamics to Low-Level Control . . . 6
8. Why Humanoid Robots Need Floating-Base Dynamics . . . 6
9. Interpreting Physical Parameters in MuJoCo and Isaac . . . 7
10. From Simulation to Hardware: Diagnose the Model From Failure Modes . . . 7
11. One-Page Summary: The Equations Worth Remembering . . . 8
References . . . 9

Course home ·简体中文 ·PDF

Item	Details
Topic	Robot Dynamics
Revision	2026-10-30
Prerequisites	Introductory kinematics, linear algebra and calculus
License	CC BY-NC-SA 4.0 ·Xiong Zeng and contributors

1. Understanding Dynamics Through a Single Grasp

> **One question guides this tutorial:** After a robot policy issues an action, why do the fingers move, why does the screwdriver stay in the grasp, or why does it suddenly slip?
>
> **The physical chain:** Policy action a_t $\longrightarrow$ actuator torque τ_t $\longrightarrow$ robot/object accelerations and contact forces $\longrightarrow$ next state s_{t+1} $\longrightarrow$ sensor observation o_{t+1} $\longrightarrow$ next action.

We use one running example throughout: **a robot arm with a dexterous hand picks up and moves a screwdriver**. The goal is to understand the physics underlying robot learning and physical simulation; no software installation or code is required. Unless stated otherwise, the first part assumes a **fixed base, rigid links, and independent joint coordinates**. Contact, actuator nonidealities, and floating bases are introduced separately.

Kinematics describes where; dynamics explains how motion happens

Imagine that a dexterous hand has located a screwdriver on a table. The arm must bring its fingers to the object, close the grasp, lift the screwdriver, and move it to a target location.

- **Kinematics** asks: Where is a fingertip at a given joint configuration? Where should the end effector move?
- **Dynamics** asks: What motor torques are required? Why does the object slip? Why does the motion lag behind a command?
- **Contact mechanics** asks: What normal and friction forces arise among the table, fingers, and screwdriver?

Crucial distinction: A target pose being kinematically reachable does not mean the motors can reach it within a specified time, or that the object can be grasped stably.

State, observation, and action are different quantities

Let $q \in \mathbb{R}^n$ denote joint positions and $\dot{q}$ joint velocities. The full physical state also includes object poses, velocities, and contact modes. A policy, however, might only receive camera images, encoder readings, and motor-current measurements.

Concept	Example in screwdriver grasping	Common misconception
State s	Joint/object velocities and actual contact mode	Not everything is directly measurable
Observation o	RGB-D image, encoders, motor currents	Not the full physical state
Action a	Desired joint position, velocity, torque, or end-effector pose	Its units and command interface matter

The first question to ask when studying a robot policy is therefore not merely "What does the network output?" but **"Which low-level controller executes that output?"**

2. Actuators: Why a Position Command Does Not Take Effect Instantly

From desired position to actual torque

If a policy outputs a desired joint position q_d, a common joint-servo model is

$$\boxed{\tau_{\text{cmd}} = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}).}$$

Here K_p converts position error into restoring torque, while K_d provides velocity damping. This is the **commanded torque computed by a controller**, not necessarily the torque the hardware actually delivers. Real actuators are constrained by motor-current/torque limits, speed limits, drive filtering, transmission friction, and delay.

For example, when a finger is already pressing against the screwdriver, asking it to close farther usually increases the servo's torque command. The finger does not necessarily move farther: contact reaction forces may prevent it.

The same policy action can behave differently on different robots

Policy action	How it is executed	Parameters to check first
Desired joint position	Internal position servo generates motor torque	K_p, K_d, saturation, control rate
Desired joint velocity	Velocity servo tracks a target speed	Acceleration limits, filtering, delay
Joint torque	Torque/current control drives the joint	Torque calibration, saturation, safety limits
End-effector pose	IK or task-space control creates joint commands	Reference frames, IK, low-level controller

One further parameter is **motor reflected inertia (armature)**. For an ideal single-stage gear train, a motor with rotor inertia I_m and gear ratio N contributes approximately $N^2 I_m$ when reflected to the joint. This is distinct from link inertia. The simple single-joint formula does not directly describe coupled transmissions or compliant tendon mechanisms.

Takeaway: A policy produces a *request*; the actuator determines how that request becomes a *physical driving force*.

3. Mass, Inertia, and Gravity: Start With One Joint

Why mass alone does not determine motion

For translation, Newton's second law gives $F = ma$. Rotation also depends on how mass is distributed relative to the rotation axis.

For a rigid body rotating about a fixed axis, the parallel-axis theorem gives

$$I_O = I_C + mr^2,$$

where I_C is the moment of inertia about the center of mass and r is the center-of-mass distance from the joint axis. Two objects with the same mass can demand different torques if their mass is distributed differently. Picking up a tool can make the robot not only "heavier" but also harder to accelerate rotationally.

A single rigid rod explains the essential physics

Consider a rigid rod moving in a vertical plane. Define the joint angle q as positive upward from the horizontal; the rod has mass m, center-of-mass distance r, and moment of inertia I_O about its pivot. Neglecting friction,

$$\boxed{I_O \ddot{q} + mgr \cos q = \tau.}$$

This equation can be read in three steps:

1. $I_O \ddot{q}$: changing angular velocity requires torque to overcome rotational inertia.
2. $mgr \cos q$: gravity creates a posture-dependent torque about the joint.
3. τ: the actuator torque must account for both effects.

When the rod is horizontal and stationary, $\dot{q} = \ddot{q} = 0$ and $\tau = mgr$ is required. When the rod is vertical, the ideal gravity torque at rest is zero. **Stationary does not imply zero motor torque**.

What changes after grasping a screwdriver?

The hand and tool have a different combined mass distribution, potentially changing total mass, center of mass, and inertia. Gains K_p, K_d that work well with an empty hand can produce lag, overshoot, or vibration while holding a tool. Check the payload model before simply retuning the gains.

4. Multi-Joint Robots: One Equation Explains the Coupling

What is fundamentally different from a single joint?

In a chain of links, moving one joint also moves other links. The effective inertia seen at a joint therefore depends on the configuration of the entire mechanism. Without contact, the standard equation is

$$\boxed{M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) = \tau.}$$

Symbol	Physical interpretation	Meaning for hands and arms
$M(q)$	Configuration-dependent mass matrix	Posture changes inertia coupling and response
$C(q,\dot{q})\dot{q}$	Velocity-dependent coupling terms	More visible during fast motion
$g(q)$	Generalized gravity forces needed to hold posture	Changes with load and configuration
τ	Joint driving torques	Limited by actuators and control interfaces

It is common to write $h(q,\dot{q}) = C(q,\dot{q})\dot{q} + g(q)$, so that $M\ddot{q} + h = \tau$. Off-diagonal entries of M generally represent inertial coupling between joints. **You do not need to derive a full two-link mass matrix before understanding this fact**.

Forward dynamics versus inverse dynamics

- **Forward dynamics**: Given $q, \dot{q}, \tau$, compute $\ddot{q}$. Physical simulation largely addresses this question.
- **Inverse dynamics**: Given $q, \dot{q}, \ddot{q}_d$, compute the torque τ needed to produce a desired acceleration. Feedforward control and model-based control often use this calculation.

$$\ddot{q} = M(q)^{-1}[\tau - h(q,\dot{q})], \qquad \tau = M(q)\ddot{q}_d + h(q,\dot{q}).$$

The matrix inverse expresses the mathematical relationship. In practice, software typically solves the corresponding linear system instead of explicitly forming an inverse.

No need to derive dozens of joints by hand

Dynamics libraries rely on recursive algorithms: **RNEA** for inverse dynamics, **ABA** for forward dynamics, and **CRBA** for the mass matrix. For robot-learning applications, first understand their inputs and outputs, then check that the model's masses, centers of mass, inertias, and joint coordinates are correct.

5. The Jacobian: Connecting Fingertip Motion and Joint Forces

How fast does the fingertip move when joints move?

For fingertip position $p = f(q)$, the Jacobian relates joint velocity to fingertip velocity locally:

$$\boxed{\dot{p} = J_p(q)\dot{q}.}$$

This is a local linear map showing how multiple joints cooperate to move the fingertip. For full six-dimensional motion, including orientation, use the corresponding spatial-velocity Jacobian with consistent reference frames and reference points.

Why does a fingertip force produce joint torque?

If an external force f acts at the fingertip, its equivalent generalized joint force is

$$\boxed{\tau_{\text{ext}} = J_p(q)^T f.}$$

The relationship follows from **virtual work**: the work of a small fingertip displacement must equal the generalized work associated with the corresponding joint displacement. During grasping, contact forces exerted by the screwdriver influence each joint through precisely this mapping.

Division of labor: IK converts a desired fingertip location into joint targets. The Jacobian relates velocities and forces. Dynamics determines the resulting motion under inertia, gravity, and contact constraints. These are complementary tools, not substitutes.

6. Contact and Friction: Why Objects Stay Grasped or Slip

Contact changes the motion the robot would otherwise make

During free motion, acceleration is determined by actuator torques and the robot's own dynamics. Once a finger touches the screwdriver, the object exerts a contact force λ. The equation becomes

$$\boxed{M(q)\ddot{q} + h(q, \dot{q}) = \tau + J_c(q)^T \lambda.}$$

Here J_c is the contact Jacobian. The key point is that **contact forces feed back into joint motion**. When contact persists without slipping, relative contact-point motion must satisfy the appropriate kinematic constraints; this no-slip assumption must not be imposed after sliding or separation begins.

A table can push, but it cannot pull

Let $\phi \geq 0$ denote the separation gap and $\lambda_n \geq 0$ the normal contact force. Ideal unilateral rigid contact satisfies

$$\phi \lambda_n = 0.$$

When the object leaves the table, $\phi > 0$ and the supporting force is zero. When it rests on the table, $\phi = 0$ and a normal force may exist. This is the **unilateral nature of contact**. An impact may also cause an abrupt change in velocity, so the simulator must handle impulses and contact-mode transitions.

Friction determines whether a grasp slips

Under a simplified Coulomb friction model, tangential and normal contact forces satisfy

$$\boxed{\|\lambda_t\| \leq \mu \lambda_n.}$$

Here μ is the coefficient of friction and λ_t is the tangential contact force. If maintaining a grasp requires a tangential force beyond this friction limit, relative sliding occurs. Grasp failure may result from insufficient grip force, low friction, changing object orientation, or inertial loads during fast acceleration—not just fingers that failed to close far enough.

Contact phase	Observable behavior	Control/learning implication
Approach	Fingertips have not yet touched the object	Adjust position and orientation
Contact onset	Normal force rises	Avoid excessive impact and squeezing
Stable grasp	Relative slip remains small	Maintain a sufficient friction margin
Sliding or release	Object drifts; contact force decreases	Correct the grasp or regrasp

Running example: Lifting a screwdriver requires success at three different levels: geometric *reachability*, actuator *grip capability*, and contact *stability*.

7. From Dynamics to Low-Level Control

Three useful physical intuitions

PD position control behaves like a virtual spring and damper: displacement from the target produces restoring torque, while rapid motion is damped. It is simple, but it does not automatically remove steady-state errors caused by gravity or payload changes.

Gravity compensation adds $g(q)$ to the PD command:

$$\tau_{\text{cmd}} = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}) + g(q).$$

This helps hold posture; it does **not** automatically solve contact and friction.

Impedance control shapes how stiff or compliant the robot feels during interaction. For example, a simplified task-space virtual spring-damper relation is

$$F_{\text{cmd}} = K_x(x_d - x) + D_x(\dot{x}_d - \dot{x}).$$

This commanded task-space generalized force can be mapped to joint torques through the Jacobian, possibly together with gravity and other dynamic compensation. F_{cmd} is a controller-generated quantity, not necessarily the actual contact force: actual contact depends on the coupled robot, object, and environment dynamics.

High-level policies and low-level controllers have distinct roles

VLA, imitation-learning, and RL policies may output target positions, increments, or torques; the low-level controller must implement the request within physical constraints. Any comparison of policies should specify action type, control period, PD gains, actuator limits, and contact model.

Full-body coordination may also require computed-torque control, model predictive control (MPC), or whole-body control (WBC). Their shared requirement is that **desired motion remain compatible with dynamics, contacts, and actuator constraints**. Detailed algorithms belong in a separate course on robot low-level control.

8. Why Humanoid Robots Need Floating-Base Dynamics

A tabletop robot arm usually has a fixed base. In contrast, the torso of a humanoid or quadruped can translate and rotate through space. Joint motors **cannot directly apply an arbitrary levitating force to the entire body in world coordinates**. The robot needs external contacts, such as its feet against the ground, for support and propulsion.

For n actuated joints, floating-base generalized velocity is typically $n + 6$ dimensional. Writing the generalized velocity as v, the dynamics can be expressed as

$$M(q)\dot{v} + h(q, v) = S^T\tau + J_c(q)^T\lambda.$$

The selection matrix S indicates that only some generalized coordinates are directly actuated. The center of mass c must also satisfy

$$m\ddot{c} = m\mathbf{g} + \sum_i f_i.$$

While standing, ground-reaction forces must approximately balance gravity. Once airborne, moving internal joints cannot

make the center of mass violate the motion dictated by external forces. This is why foot-ground contact matters so much in whole-body control and legged-robot RL.

9. Interpreting Physical Parameters in MuJoCo and Isaac

Understand the engines' roles before comparing results

MuJoCo is widely used to construct and study articulated rigid-body, actuator, and contact models, making it convenient to isolate physical mechanisms.

Isaac Sim supports robot and scene simulation. **Isaac Lab** builds training environments, observations, actions, rewards, and large-scale parallel learning workflows on top of Isaac Sim. Isaac Lab is not a completely independent physics engine.

The tools serve different purposes, but **matching URDF files or visual geometry does not guarantee matching dynamics**. When comparing the same robot across engines, check the *loaded physics parameters and solver settings*, not merely the asset filenames.

Minimal parameter checklist for dexterous hands

Layer	Check first	Typical symptoms of a wrong setting
Rigid bodies	Mass, center of mass, inertia tensor, inertial frames	Incorrect acceleration or swing response
Joints	Axes, limits, damping, friction	Wrong direction or inaccurate low-speed motion
Actuators	Reflected inertia, gains, torque/velocity limits	Vibration, lag, saturation
Contact	Collision shapes, friction, solver parameters	Penetration, slipping, different impact response
Timing	Physics step, controller period, policy period	Unstable fast contacts or mismatched delay

Keep three time scales separate

The physics integration step Δt_{phys}, low-level control period Δt_{ctrl}, and policy-action period Δt_{π} are **usually not the same**. A policy may issue a position target once, the position servo may update several times before the next policy action, and the physics solver may integrate at an even smaller time step.

Do not interpret "a 20 Hz policy" as "the motor controller updates only 20 times per second." Smooth-looking rendered video also does not prove that collision resolution is accurate. Report all three rates and how commands are held between updates.

10. From Simulation to Hardware: Diagnose the Model From Failure Modes

A reliable order for diagnosis

Begin with **contact-free motion** to check joint directions, gravity, inertia, and actuators. Next test **one stable contact** to evaluate friction and contact response. Only then test **complete grasping and transfer**, including contact-mode transitions and closed-loop delay. Do not use extreme randomization to hide basic modeling errors from the outset.

Failure	Check first	Next action
Empty-hand tracking lags	Controller rate, K_p/K_d, velocity/torque limits	Compare measured joint step responses

Failure	Check first	Next action
Hand vibrates with a tool	Payload mass, center of mass, inertia, gains	Calibrate payload and control parameters
Grasp succeeds in sim but slips on hardware	Fingertip friction, contact shapes, grip force	Estimate contact parameters; randomize moderately
Impact force is too large	Filtering, approach velocity, collision parameters	Inspect pre-impact speed and sampling rates
Vision-guided grasp is misaligned	Camera extrinsics, vision latency, occlusion, estimator	Separate perception error from dynamics error

Randomization should have a physical basis

Domain randomization should represent plausible real-world uncertainty, rather than simply maximizing parameter ranges. Mass, center of mass, and inertia are related: moving a payload can also change inertia. For dexterous hands, useful initial targets include object mass/center of mass, joint damping, actuator gains, delay, and fingertip friction. Expand ranges based on hardware measurements rather than guessing.

Final validation criterion: For the same command sequence, do simulation and hardware exhibit comparable free-motion responses, contact onset, sliding, and payload effects? Different engines need not agree frame by frame, but the important physical behaviors should be explainable and testable.

11. One-Page Summary: The Equations Worth Remembering

Physical question	Key equation	Interpretation
How does a position target drive a joint?	$\tau_{\text{cmd}} = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q})$	Servo converts error into commanded torque
How does torque cause motion?	$M\ddot{q} + h = \tau$	Inertia and nonlinear effects determine acceleration
How does joint velocity map to fingertip velocity?	$\dot{p} = J\dot{q}$	Jacobian maps velocities
How does fingertip force affect joints?	$\tau_{\text{ext}} = J^T f$	Jacobian transpose maps forces
How does contact affect motion?	$M\ddot{q} + h = \tau + J_c^T \lambda$	Reaction forces modify acceleration
Why can a grasp slip?	$\|\lambda_t\| \le \mu\lambda_n$	Available friction is bounded
How does a humanoid balance?	$m\ddot{c} = m\mathbf{g} + \sum_i f_i$	External forces govern center-of-mass motion

Learn what each term means and when each model applies. Full derivations of recursive multibody algorithms are not a prerequisite for practical robot learning.

Extension: Physical simulation versus learned dynamics models

A physics simulator computes $s_{t+1} = F_{\text{sim}}(s_t, a_t)$ from masses, inertias, actuators, and contact models. A learned dynamics model instead fits $\hat{s}_{t+1} = f_\theta(s_t, a_t)$ from data. The learned model may offer easier differentiation or faster predictions, but **accurate next-state prediction does not imply accurate gradients with respect to actions**, especially near transitions among impact, sticking, sliding, and separation. If a learned model is used to optimize a policy, evaluate short-horizon state prediction and gradient-based control quality separately. SuperMimic provides one research example of this approach.

References

1. Modern Robotics — Chapter 8: Dynamics of Open Chains — articulated-body dynamics and forward/inverse dynamics.
2. Modern Robotics — Chapter 11: Robot Control — motion control.
3. Featherstone — Spatial Vector Resources — RNEA, CRBA, and ABA.
4. MuJoCo — Computation — contact, constraints, actuators, and numerical solution.
5. Isaac Lab — Articulations — multibody assets and models.
6. Isaac Lab — Actuators — actuator models.
7. MIT — Underactuated Robotics: Multibody Dynamics — floating bases and underactuation.
8. MEE5114 — Advanced Control for Robotics — additional structured reading.
9. SuperMimic — learned surrogate dynamics as a research example.